



UNIVERSITY OF PISA
MASTER'S DEGREE IN CYBERSECURITY

TIMESTAMPING SERVICE
FOUNDATION OF CYBERSECURITY

Authors
Leonardo Corsini
Elena Palandri

Academic year 2025/2026

Contents

1	Introduction	1
1.1	Description	1
1.2	Security Requirements	1
2	Specifications and Design	3
2.1	Project Structure and Files	3
2.2	System Architecture	3
2.3	Key Infrastructure	4
2.4	Cryptographic Primitives and Secure Choices	4
2.5	Security of the Channel	5
2.6	Handshake	5
2.7	Application Messages and Local Hashing	5
3	Messages' Format	7
3.1	Framing of the Packets	7
3.2	Handshake Messages (Cleartext)	7
3.2.1	ClientHello (MSG_TYPE_HANDSHAKE)	8
3.2.2	ServerHello (MSG_TYPE_HANDSHAKE)	8
3.3	Application Messages (Encrypted)	8
3.3.1	User Authentication (Login)	8
3.3.2	Timestamp Operation	8
3.3.3	Balance Operation	9
3.4	The Timestamp Token Format (.tsr)	9
4	Sequence Diagrams and Protocol Workflows	10
4.1	Handshake and Server Authentication	10
4.2	User Authentication (Login)	11
4.3	Timestamping and Balance Operations	12
5	Demo and Test Scenarios	15
5.1	Successful Timestamping	15

5.2	Unsuccessful Timestamping (Empty Balance)	16
5.3	Offline Verification of a Timestamp	16
5.4	Balance Request	16
6	Conclusions	18
6.1	Limitations and Possible Improvements	18

CHAPTER 1

Introduction

1.1 Description

A Timestamping Service (TSS) provides a trusted third-party service that provides a legally valid cryptographic signature on data (such as documents, codes, or logs). The Time Stamping Authority (TSA) proves through the signature that the data existed on a specific date and time. To preserve the confidentiality of the user's data, the original document is not transmitted; the client calculates the hash of the data locally and sends it as a fixed-length value to the TSA. Once the request is received, the server combines the received hash with the timestamp and digitally signs the resulting package.

The resulting signed timestamp token is structured as <hash, time, signature> and serves as cryptographic evidence of the document's existence at the specified time. This token can be verified independently and offline by anyone holding both the TSA's public key and the original document. This allows one to prove that neither the document nor the timestamp has been maliciously altered.

1.2 Security Requirements

As the project requirements, we assume that the users have already registered and paid the fee. Upon registration, the user receives a username and a password binding their identity to a valid account.

Users interact with the TSS through a secure channel that must be established before issuing operations. The channel must follow the following security requirements:

- Perfect Forward Secrecy (PFS): Ensures the compromise of long-term keys does not compromise past session keys.

- Integrity: Ensures that data is accurate, complete, and remains unaltered or uncorrupted by unauthorized users throughout its lifecycle.
- No-Replay: Ensures that an adversary is not able to intercept a transaction or a message and to replay it to deceive the system.
- Non-Malleability: Ensures a ciphered system or message cannot be altered by adversaries without detection. If an encrypted or signed message is intercepted, an attacker cannot manipulate the data to create a new, meaningfully related message, even without knowing the original contents.

To protect communications and operations, the service uses two distinct pairs of cryptographic keys:

- (pubKc, privKc), dedicated to the authentication of the communication channel
- (pubKts, privKts), dedicated to digitally sign the timestamp

Requirements also include mutual authentication: On one side, the client must authenticate the server using the pubKc during the handshake to prevent Man-in-the-Middle attacks. On the other side, the user must authenticate to the service through its credentials.

Finally, the server must implement a rigorous check on the volume of paid timestamps: the timestamping operation must fail and return an error if the authenticated user has finished their prepaid volume of available timestamps.

CHAPTER 2

Specifications and Design

2.1 Project Structure and Files

The software architecture has the following modules:

- `server.py`: The main multi-threaded server application. It accepts concurrent TCP connections, negotiates the secure tunnel, authenticates clients, and signs the timestamp requests.
- `client.py`: The client application, structured as a Finite State Machine (FSM) that establishes the secure channel, securely transmits credentials, hashes local documents, and stores the resulting tokens.
- `shared.py`: A utility module that standardizes the binary framing of network messages and encapsulates the AES-256-GCM cryptographic logic within the `SecureChannel` class.
- `db.py`: The database management interface.
- `gen_keys.py`: It generates the mandatory long-term key pairs to sign both channel and timestamp.
- `verify.py`: Utility used offline to verify the timestamp in `.tsr` format.

2.2 System Architecture

The project, implemented in Python, creates a Client-Server architecture and operates through Transmission Control Protocol (TCP) sockets. The server utilizes a multi-threading approach through the `threading` module. Every connection is sent to the

`ClientHandler` thread, allowing the server to manage multiple users at the same time, just as it would be in the real world.

To manage user information in a secure way, the project has a local SQLite3 database that stores user credentials and tracks the volume of consumed timestamps (`nc`) and remaining ones (`nr`). To prevent race conditions and double-spending of tokens during concurrent requests, database access (queries and updates) is allowed only to one user at a time. This guarantees the program is protected by isolation and mutual exclusion mechanisms and is made by the `threading.Lock()`.

2.3 Key Infrastructure

The service uses two different long-term key pairs both logically and functionally separated:

- **Channel Authentication Key** (`pubKc`, `privKc`): This key pair is only used to establish the secure channel. The server uses `privKc` to sign the handshake message, allowing the client to verify the server's identity and to prevent Man-in-the-Middle (MitM) attacks. The client uses `pubKc` to verify this signature and authenticate the service.
- **Timestamp Signing Key** (`pubKts`, `privKts`): The server uses `privKts` just to apply digital signatures on the timestamp tokens. Then, the client uses `pubKts` to validate the token's authenticity.

2.4 Cryptographic Primitives and Secure Choices

The architecture uses the cryptographic primitives contained in the `cryptography.hazmat` Python library to fulfill the project's security requirements:

- **ECDHE (SECP256R1)**: Elliptic Curve Diffie-Hellman Ephemeral is used for the key exchange. It generates one-time private keys for every session, guaranteeing PFS. We chose the SECP256R1 (P-256) curve because it provides a strong 128-bit security level, ensures fast computations and small key sizes (65 bytes), especially compared to traditional RSA.
- **HKDF-SHA256**: The raw shared secret derived from ECDH is not uniformly distributed, and it wouldn't be safe to be used directly as an encryption key. To solve this, we use a Hash-based Key Derivation Function (HKDF) to make the shared secret into a 32-byte sequence uniformly distributed. The nonces of the client and the server are concatenated and used as the HKDF *salt*. This binds the derived keys to the current unique session.
- **AES-256-GCM**: The Galois/Counter Mode (GCM) is used to protect the communication channel. AES-GCM provides confidentiality, data integrity, and non-malleability.
- **Salted Password Hashing**: To protect user credentials, passwords are not stored in plain text. The SQLite database stores a salted hash of the password generated through a combination of PBKDF2-HMAC-SHA256 with 200,000 iterations.

During the login phase, the server verifies the credentials using a constant-time comparison to prevent side-channel timing attacks.

2.5 Security of the Channel

The custom secure channel satisfies all mandatory security requirements. To summarise, Table 2.1 represents on the left each requirement and on the right the implemented mechanism.

Requirement	Cryptographic Mechanism
Perfect Forward Secrecy	Ephemeral ECDHE key exchange
Server Authentication	ECDSA signature over the STS transcript
Confidentiality	AES-256 encryption
Integrity & Non-Malleability	GCM authentication tag (MAC)
No-Replay	Random Nonces (Handshake) + Sequence Numbers (GCM)

Table 2.1: Mapping of security requirements to implemented mechanisms.

To mitigate Reflection Attacks, the protocol implements **Directional Initialization Vectors (IVs)** within the AES-GCM `SecureChannel`. Specifically, the 12-byte IV is constructed by embedding the sender's role in the very first byte (e.g., 0 for the Server and 1 for the Client).

2.6 Handshake

The client and server execute a handshake over the cleartext socket to establish the secure channel.

1. **Client:** The client generates a 32-byte cryptographically secure random nonce (N_c `client_nonce`) and an ephemeral key pair. It sends the uncompressed public key ($Epub_c$) and the nonce to the server.
2. **Server:** The server generates its own ephemeral key pair and a 32-byte nonce (N_s `server_nonce`). It then computes an ECDSA signature using `privKc` over the exact transcript of the exchange: ($Epub_c \parallel N_c \parallel N_s \parallel Epub_s$). The server replies with $Epub_s$, N_s , and the signature.
3. **Verification and Derivation:** The client verifies the signature using the pre-shared `pubKc`. If valid, both parties compute the ECDH shared secret and pass it through the HKDF.

A visual representation of the scheme can be found in Figure 4.1

2.7 Application Messages and Local Hashing

Once the handshake is completed, the state machine transitions to the secure phase. All subsequent packets are framed with a Message Type header and encapsulated within the AES-GCM.

- **Encrypted Login (MSG_TYPE_AUTH):** The client sends the username and password as a single message, where the payload inside the encrypted tunnel is formatted as:

Lu | Lp | Username | Password

Lu and Lp are both 4-byte data, representing username length and password length. This way, credentials never travel in cleartext, and it is possible to calculate where the username finishes and the password begins.

- **Timestamping (MSG_TYPE_TIMESTAMP):** The original document is never uploaded to the server. This preserves confidentiality and database saturation. The client locally computes the SHA-256 of the file (dividing it into chunks to support extremely large files) and sends only the 32-byte hash. Once the server checks and validates the user's balance, it concatenates the hash with the current Unix time (packed as an 8-byte big-endian integer), and signs all with `privKts`.
- **The Token (.tsr file):** The client receives the response and saves the cryptographic proof as a JSON file containing the hash, time, and hexadecimal signature. This format allows offline verification.
- **Balance (MSG_TYPE_BALANCE):** The client securely requests its current quota, and the server queries the SQLite database to return the exact number of consumed and remaining tokens.

CHAPTER 3

Messages' Format

3.1 Framing of the Packets

TCP is a stream-based protocol. This means that it does not preserve message boundaries. To ensure the extraction of the payload is performed correctly from the stream and prevent fragmentation issues, all network messages in the system use a 5-byte custom header, handled by the `pack_message` and `unpack_message` functions. The 5-byte custom header is composed of 1 byte for the type of the message and 4 bytes for the length of the payload.

The framing structure is defined as follows:

```
MSG_TYPE (1 byte) | PAYLOAD_LENGTH (4 bytes) | PAYLOAD (N bytes)
```

- **MSG_TYPE:** The first byte specifies the operation to be performed (e.g., HANDSHAKE, AUTH (authentication), TIMESTAMP, BALANCE).
- **PAYLOAD_LENGTH:** The subsequent four bytes encode the length of the payload as an unsigned 32-bit integer in big-endian network byte order.

This mechanism allows the receiver to safely read the exact amount of expected bytes from the socket, mitigating buffer overflow vulnerabilities and desynchronization.

3.2 Handshake Messages (Cleartext)

Before the AES-GCM channel is established, the initial handshake executes in cleartext. However, the integrity and authenticity of these messages are cryptographically guaranteed by the server's signature.

3.2.1 ClientHello (MSG_TYPE_HANDSHAKE)

The client initiates the connection by sending a 97-byte payload:

```
Epub_c (65 bytes) | Client_Nonce (32 bytes)
```

The `Epub_c` is the client's ephemeral public key generated on the SECP256R1 curve, serialized in uncompressed format (which requires 65 bytes). The `Client_Nonce` is a 32-byte cryptographically secure random value used to prevent replay attacks.

3.2.2 ServerHello (MSG_TYPE_HANDSHAKE)

The server replies with its own ephemeral parameters and the authentication signature:

```
Server_Nonce (32 bytes) | Epub_s (65 bytes) | ECDSA Signature  
(variable)
```

The signature is computed by the server over the exact transcript of the exchanged keys and nonces. The client extracts the components using precise byte offsets to verify the server's identity before deriving the session key.

3.3 Application Messages (Encrypted)

Once the handshake completes and the state machine transitions to the `STATE_LOGIN` phase, all subsequent payloads and the 5-byte framing headers are fully encrypted and authenticated using AES-256-GCM through the `SecureChannel` wrapper.

3.3.1 User Authentication (Login)

To authenticate, the client sends a `MSG_TYPE_AUTH` request. The plaintext payload inside the encrypted tunnel is formatted as:

```
Lu | Lp | Username | Password
```

`Lu` and `Lp` are both 2-byte data, representing username length and password length. The server responds with a `MSG_TYPE_AUTH_SUCCESS` to grant access or `MSG_TYPE_AUTH_FAILED` to inform of a failed login.

3.3.2 Timestamp Operation

The timestamping operation is composed by:

- Request (`MSG_TYPE_TIMESTAMP`): Instead of transmitting the file, the client locally computes its SHA-256 digest and sends only the resulting 32-byte hash as the payload.
- Response: If the user has sufficient balance, the server responds with a payload structured as:

```
Unix Time (8 bytes) | ECDSA Signature (variable)
```

The time is packed as an 8-byte unsigned integer in big-endian format.

3.3.3 Balance Operation

The client requests its quota status by transmitting a `MSG_TYPE_BALANCE` message with an empty payload. The server queries the SQLite database and replies with the user's data, typically formatted as two 4-byte integers representing the consumed tokens and the remaining available tokens. We assumed that no user can buy more than 4.294.967.295 tokens.

3.4 The Timestamp Token Format (.tsr)

Upon a successful timestamp request, the client must store the cryptographic proof for future independent verification. The client software structures this proof as a human-readable JSON file (typically saved with a `.tsr` extension) containing three distinct fields:

- `hash`: The SHA-256 digest of the original document, stored as a hexadecimal string.
- `time`: The 64-bit Unix timestamp integer returned by the server.
- `date`: The timestamp converted to a human-readable format (Y-m-d H:M:S).
- `signature`: The server's ECDSA signature, converted to a hexadecimal string.

This JSON format was chosen because it simplifies the debugging process and allows easy inspection by the user.

Sequence Diagrams and Protocol Workflows

This chapter shows the chronological flow of operations between the client and the server. The communication lifecycle is controlled by a state machine implemented in both the client and server (`STATE_HANDSHAKE`, `STATE_LOGIN`, `STATE_READY`). This design choice strictly prevents state-jumping attacks, ensuring that no application data is exchanged before a secure channel is established and the user is authenticated.

4.1 Handshake and Server Authentication

Figure 4.1 illustrates the handshake protocol execution.

This phase corresponds to the `STATE_HANDSHAKE` status and operates over the cleartext TCP socket. The protocol works as follows:

1. **Ephemeral Key Generation:** The client generates a new 32-byte nonce (N_c) via the `os.urandom` cryptographic pseudo-random number generator and an ephemeral SECP256R1 key pair. The public key and the nonce are sent to the server.
2. **Server Authentication:** The server generates its own ephemeral parameters (N_s and E_{pub_s}). To prevent MitM attacks, the server signs the exact transcript of the exchange using its long-term channel private key (`privKc`).
3. **Verification and PFS:** The client verifies the signature using the pre-shared `pubKc`. The shared secret is then calculated using the ECDH algorithm. Finally, the HKDF calculates the symmetric keys using the concatenated nonces ($N_c \parallel N_s$) as salt. This rigorously guarantees PFS.

Upon successful completion, both entities instantiate the `SecureChannel` (AES-256-GCM) and transition to the `STATE_LOGIN` phase.

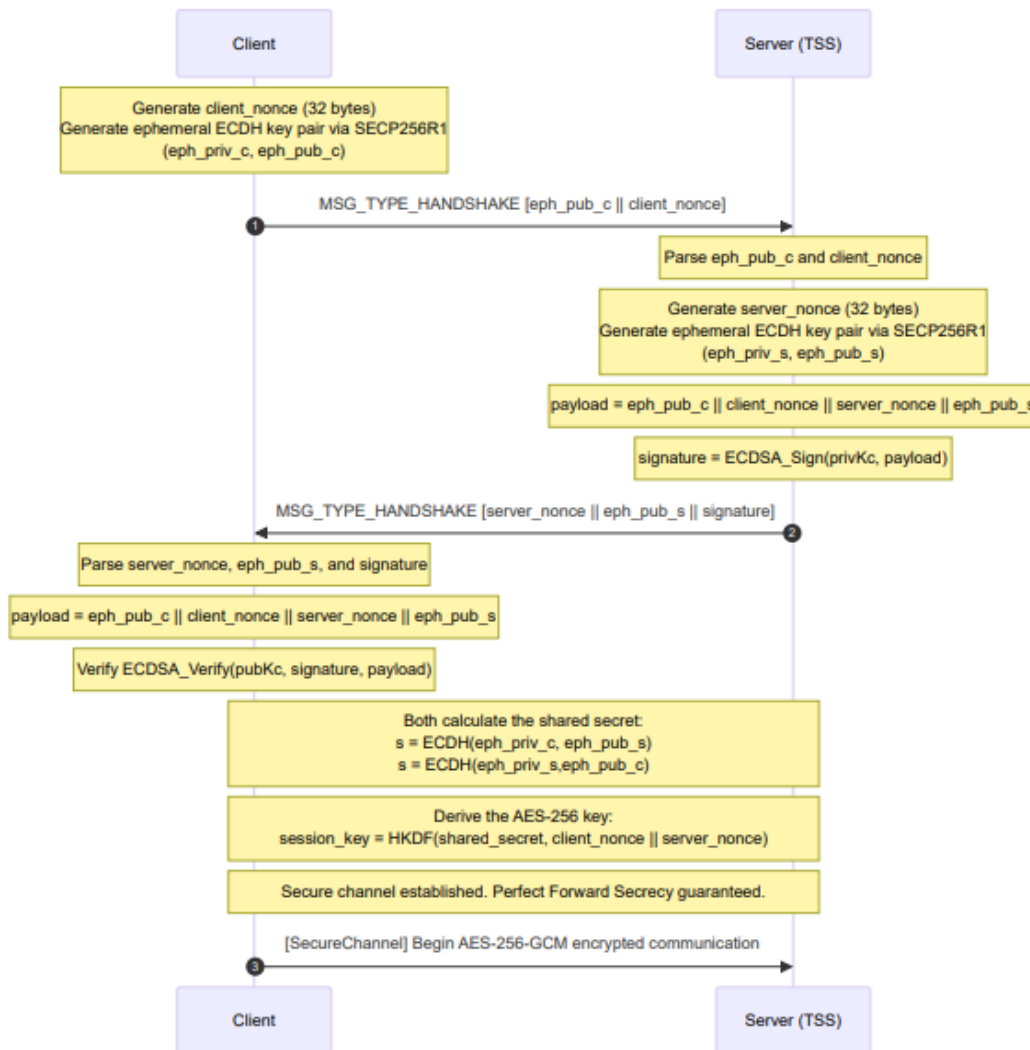


Figure 4.1: Sequence diagram of the Handshake protocol.

4.2 User Authentication (Login)

Figure 4.2 models the encrypted transmission of user credentials. This phase ensures that user identities are verified without ever exposing passwords to network eavesdroppers.

Workflow and Security Choices:

1. **Credential Encapsulation:** The client prompts the user for credentials, formats them using a 4-byte header containing the length of the username (2-bytes) and the password (2-bytes), chained with the credentials (`lu | lp | username | password`), and encrypts the payload through the `SecureChannel`.
2. **Side-Channel Resistance:** Upon receiving the payload, the server unpacks the message and queries the SQLite database. To prevent timing side-channel attacks, the server computes the hash of the received password (using the user's unique

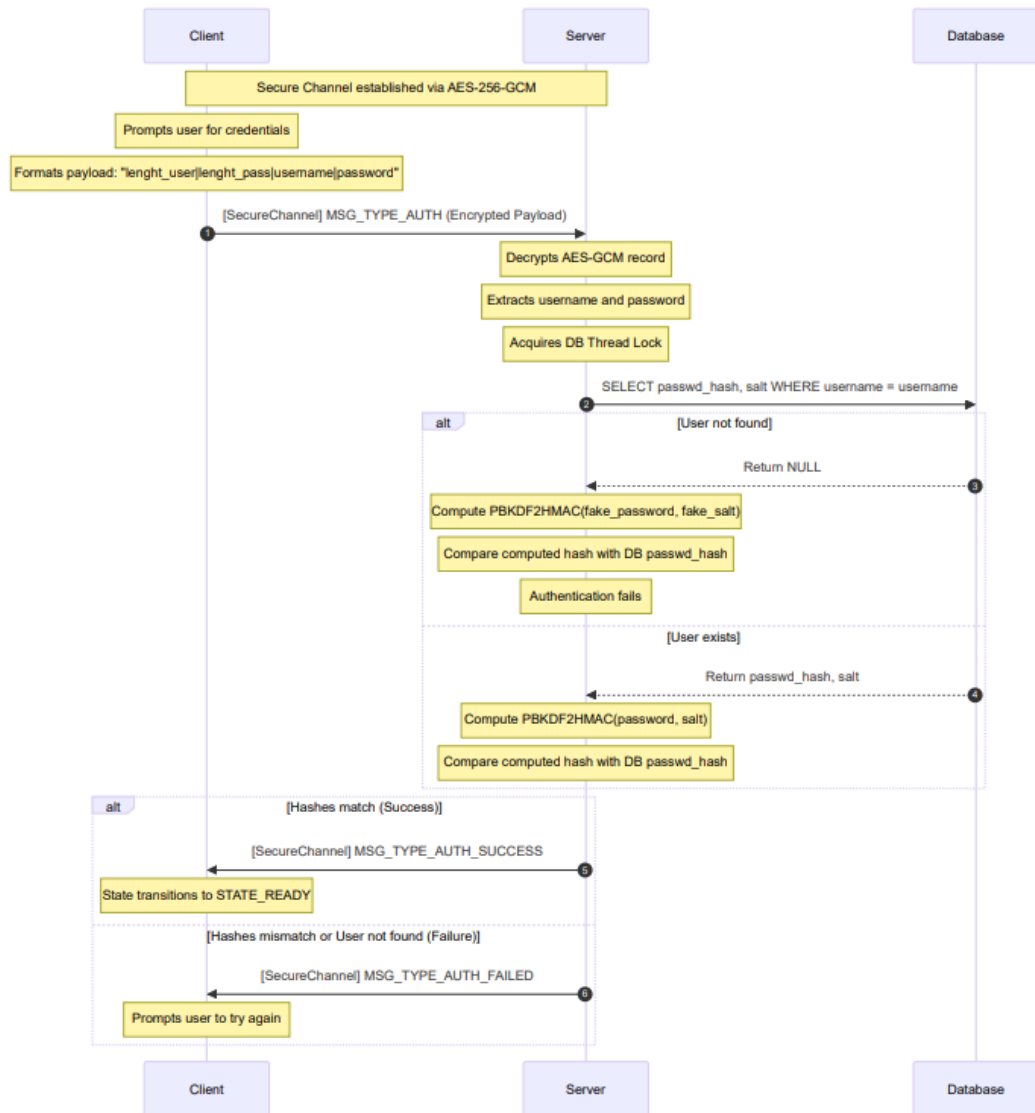


Figure 4.2: Sequence diagram of the Encrypted Login Phase.

salt and the PBKDF2 algorithm) and compares it against the stored hash using a constant-time comparison function (`kdf.verify()`). Additionally, if the username doesn't exist in the database, the server will use the method `verify_hash()` anyway, preventing inference on server timing response.

3. State Transition: If the credentials are valid, the server replies with a message `MSG_TYPE_AUTH_SUCCESS`, and both parties transition to the `STATE_READY` phase.

4.3 Timestamping and Balance Operations

Once in the `STATE_READY` phase, the client can request the core services of the TSS. Figure 4.3 illustrates the application logic, which strictly enforces quota management and data privacy.

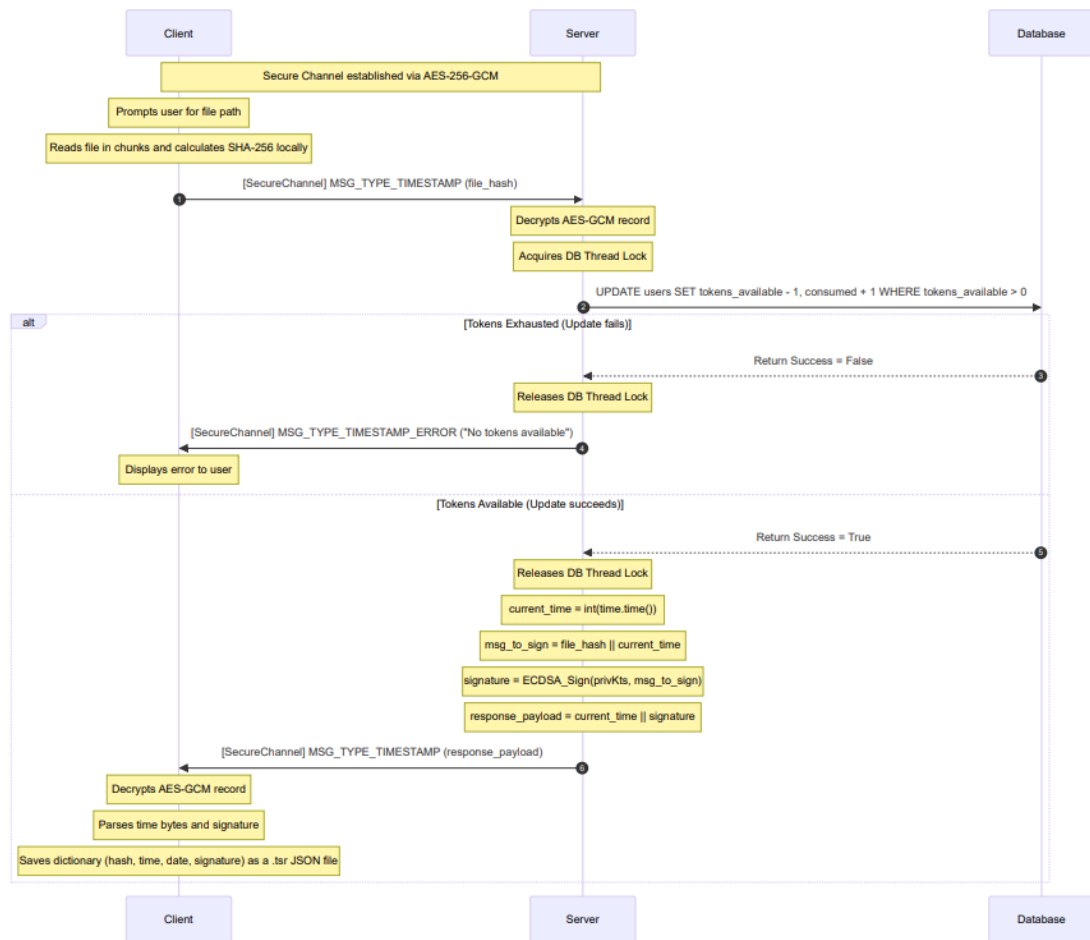


Figure 4.3: Sequence diagram of the Timestamp requests.

Workflow and Security Choices:

1. **Memory-Efficient Local Hashing:** When the user requests a timestamp for a file, the client does not upload the document. Instead, it reads the file in 8192-byte chunks and computes the SHA-256 digest locally. This prevents RAM exhaustion for large files and ensures the total privacy of the user's data. Only the 32-byte hash is transmitted through the encrypted tunnel.
2. **Concurrency and Race Conditions:** Upon receiving the hash, the server retrieves the user's balance from the SQLite database. Since the server operates in a multi-threaded environment, the balance check and the subsequent token deduction are wrapped inside a `threading.Lock` (Mutex). This guarantees atomicity and prevents double-spending or race conditions if multiple requests are issued simultaneously.
3. **Token Generation:** If the balance is sufficient, the server retrieves the current Unix time, packs it into an 8-byte big-endian format, and concatenates it with the hash. The `privKts` key is then used to sign this bundle.
4. **Offline Verification Readiness:** The server returns the time and the signature to

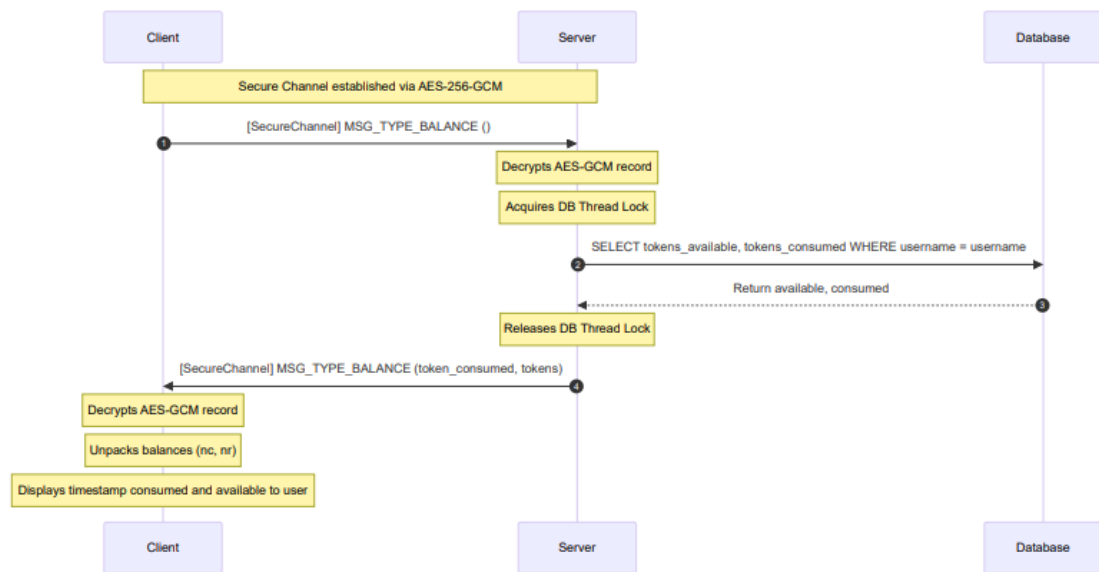


Figure 4.4: Sequence diagram of the Balance requests.

the client, which formats them into a `.tsr` JSON file alongside the original hash, providing the user with cryptographically verifiable, offline proof of existence.

CHAPTER 5

Demo and Test Scenarios

This chapter presents the execution logs of the implemented system, verifying its compliance with the project requirements. The scenarios show the service operating through the established AES-GCM secure channel.

5.1 Successful Timestamping

In this scenario, a registered user with a positive token balance requests a timestamp. The client application locally computes the SHA-256 hash of the selected file and securely transmits it to the server. The server queries the SQLite database, verifies the available balance, deducts one token under a thread lock to prevent race conditions, and generates the signature using the dedicated timestamping private key. The client receives the response and saves the cryptographic token in a local JSON file.

```
Handshake completed.
Username: alice
Password: 123
Successful Login!

--- MENU ---
1. Request Timestamp
2. Balance
3. Logout
Type your choice: 1
Insert the path of the file to timestamp: C:\Users\Improve\Desktop\Progetti\Progetti Università\TimeStampingService\work_plan.md
Calculated Hash, to certify: b67c23db4c99c73cdaa770a0fd3125d54e30f05e61ec22aecad7115fe557fe56
Timestamp received and saved in C:\Users\Improve\Desktop\Progetti\Progetti Università\TimeStampingService\work_plan.md.tsr
```

Figure 5.1: *Client view of a successful timestamp.*

5.2 Unsuccessful Timestamping (Empty Balance)

This scenario demonstrates the server's quota management. An authenticated user attempts to request a timestamp, but their prepaid volume is exhausted. The server's database transaction safely prevents the operation, and a formalized error is returned through the encrypted tunnel.

```
Handshake completed.
Username: bob
Password: 123
Successful Login!

--- MENU ---
1. Request Timestamp
2. Balance
3. Logout
Type your choice: 1
Insert the path of the file to timestamp: C:\Users\Improve\Desktop\Progetti\Progetti Università\TimeStampingService\work_plan.md
Calculated Hash, to certify: b67c23db4c99c73cdaa770a0fd3125d54e30f05e61ec22aecad7115fe557fe56
Error from server: No tokens available
```

Figure 5.2: *Client view of an unsuccessful timestamp.*

5.3 Offline Verification of a Timestamp

The verification tool operates entirely offline, proving that the architecture does not require the server for validation. The verification module parses the JSON token, recalculates the document's hash, and uses the server's public timestamping key to independently verify the ECDSA signature.

```
Insert the path of the file to verify: C:\Users\Improve\Desktop\Progetti\Progetti Università\TimeStampingService\work_plan.md
Insert the path of the timestamp file: C:\Users\Improve\Desktop\Progetti\Progetti Università\TimeStampingService\work_plan.md.tsr
Timestamp is valid and signature is verified.
```

Figure 5.3: *Client view of a timestamp verification.*

5.4 Balance Request

The user requests their current quota. The server queries the SQLite database and securely returns the exact count of consumed and remaining timestamps.

```
Handshake completed.  
Username: bob  
Password: 123  
Successful Login!  
  
--- MENU ---  
1. Request Timestamp  
2. Balance  
3. Logout  
Type your choice: 2  
Timestamp consumed: 10, Timestamp available: 0
```

Figure 5.4: *Client view of a balance request.*

CHAPTER 6

Conclusions

The project implements a secure Timestamping Service (TSS) and all requested security requirements: Perfect Forward Secrecy, integrity, no-Replay, and non-Malleability.

In particular, as Table 2.1 shows, we have seen how Perfect Forward Secrecy is obtained with Elliptic Curve Diffie-Hellman, the server authentication is guaranteed with ECDSA, confidentiality, integrity, and non-malleability are given by AES-256-GCM, and no-replay is obtained through the Handshake mechanism and GCM. Furthermore, the implemented Timestamping Service successfully provides a secure and trustworthy service. By computing the document's hash locally in chunks, the client software ensures that sensitive data never leaves the user's machine. This mechanism firstly prevents security issues, secondly, memory exhaustion, and thirdly, buffer overflow. Also, the implemented SQLite database and thread locks prevent concurrency problems and double-spending of tokens.

6.1 Limitations and Possible Improvements

Although the project is already secure, there are a few concepts that would be interesting to pursue and refine, especially in a real-world Timestamping Service:

- **Database Scalability:** The system uses a simple local SQLite file for credential storage. An actual service would require a robust Relational Database Management System (RDBMS) to handle large data and high-throughput concurrent transactions seamlessly.
- **Public Key Infrastructure (PKI):** The current implementation relies on pre-shared public keys (pubKc, pubKts). A formal Timestamping Service would require a formal Certification Authority (CA) and X.509 certificates to distribute the keys.

- **User Management:** The user registration process is only simulated. Future developments should implement an enrollment service that supports remote registration and payment processing.
- **Transition to Post-Quantum Cryptography:** As the new post-quantum algorithm progress, it will be necessary to improve the security of the program to prevent the attack "Store now, decrypt later". A possible defense mechanism for the ECDHE algorithm could be its substitution with the hybrid Key Encapsulation Mechanism (KEM) combining old standard as X25519 with post-quantum ones, such as ML-KEM (Kyber). Another interesting defense mechanism for the Timestamp is Composite Signatures that prevent the falsification of the certification. To ensure the legality of the token produced by the TSS, we could use post-quantum signatures such as ML-DSA (Dilithium) or SLH-DSA (SPHINCS+).
- **Padding for credentials:** AES-GCM acts as a stream cipher, leaking the exact length of the plaintext. For sensitive plaintext that contain password it would be more secure to add random padding before the encryption.
- **Memory Wiping:** RAM containing passwords or private keys must be overwritten after use. Python's automated garbage collection and string immutability make memory wiping very difficult. For this reason, C, C++ or RUST programming languages would be considered a safer choice.